Reply to S. Danilov's comments on the mEVP communication study

N. Koldunov, 6 August 2026

We built the variant you describe, and it changes the study's conclusion. Written as you write it — the ring computation folded into the loops that already exist rather than into kernels of its own — the wide halo removes **37 to 58% of the sea-ice cost** across three meshes, against 10 to 28% for our version of the same scheme: **the rewriting multiplies the saving by two to five**. The paired runs compute the same ring and put identical message and byte counts on the wire; they differ by 360 kernel launches per time step. Our earlier conclusion, that exactness costs most of the saving, was measuring our implementation.

change of the sea-ice cost, K=8, all legs per mesh in one allocation	CORE2 1 node / 4 GPUs	fArc 4 / 16	DARS 8 / 32
wide halo, standard form, our kernels	−11.4%	−10.0%	−24.3%
wide halo, divergence form, our kernels	+1.9%	−6.1%	−28.4%
wide halo, standard form, your loops	**−57.0%**	**−36.8%**	**−43.4%**
wide halo, divergence form, your loops	**−58.2%**	**−43.3%**	**−50.1%**
same, as a share of the whole model step	−14.4%	−9.6%	−12.7%

And the saving grows with the number of GPUs. We swept node counts, 1 to 16 on both backends. On GPU the sea-ice cost of the standard scheme does not scale at all — it is flat or rising with node count on every mesh (CORE2 15.6, 15.0, 22.9, 24.4 ms at 1, 2, 4, 8 nodes) — so adding GPUs buys nothing on the ice. Written as widened loops the wide halo lowers that cost and partly restores its scaling: in your divergence form the ice cost *falls* where the standard scheme's rises, 16.9, 15.0, 13.5 ms on fArc at 2, 4 and 8 nodes. The saving is therefore −40 to −52% of the ice on fArc from 1 to 8 nodes and −31 to −51% on DARS from 2 to 8, and the table above, taken at one node count per mesh, understates it.

Giving up the owner's summation order costs what you predicted: after a time step of 120 sub-cycles the ice velocity differs from the exact wide halo by 8 to 15 units in the last place. The 60-day test of the domain decomposition, which replaces the bit-identity we gave up, passes: the boundary-to-interior ratio is 0.836 for ice thickness against 0.854 for two identical runs, and 1.196 for ice velocity against a control of 1.325 — at or below its own control on every field.

Point 1, the auxiliary fields. We instrumented the per-step exchange to report, field by field, the difference between the owner's arriving value and the value the receiving process already holds. Nine of the eleven arrive *exactly* equal — ice velocity, concentration, thickness, snow, ocean velocity, wind stress — because FESOM's own halo exchange has already put them there, as you said. The two right-hand sides do not: they are assembled and area-scaled over owned nodes only, so a halo node has no correct value at all. But the redundancy stops at the first ring, and rings $2 \dots K$ have no other source, so what could be removed is 37% of that exchange at K=2 and 17% at K=4. Since the exchange is 5.4% of the data the scheme moves and one message per neighbour per step, removing it would gain under one percent of the traffic and no messages. We have not implemented the trimmed version, and that is why.

Point 1, the kernel structure. This is the part we could not bound before, and it was much the larger. Our wide halo adds 362 kernel launches per model step, each on a slower launch path than the owned kernel it duplicates: the ring kernels carry enough array descriptors to cross a threshold in Kokkos above which arguments are staged through constant memory. The gap between consecutive launches is 14.3 μs for those against 4.4 μs for the owned kernels. Per step the ring kernels cost 5.9 ms of inter-kernel idle against 2.5 ms of arithmetic.

Your reformulation now wins almost everywhere, and we had been measuring past it. With ring kernels the standard form beat the divergence form by 13 points of ice cost on CORE2 and 4 on fArc, and our previous note decomposed that gap into a message term and a byte term. With the ring folded into the existing loops *the divergence form leads at all three meshes* — by 1.3, 6.5 and 6.7 points, ordered by how much mesh each GPU holds, which is the behaviour a halved nodal message should produce. It is ahead at every K we tried as well, and at every node count in the sweep but one — fArc on two GPU nodes, where the standard form is 0.7 points ahead, outside the repetition spread there. So it is a strong tendency rather than a law. The launch structure was not creating the difference between the two forms; it was hiding it, and it fell hardest on yours because its ring gather was the most expensive of the ring kernels. On CPU the same rewriting also favours your form, where there are no launches at all: folding the loops removes a recomputation the divergence form was doing about six times over, worth 3 to 15 percentage points of the ice, while in the standard form it costs 18 to 30. Neither is profitable on CPU in either writing, at any of the three core counts we tried.

Point 2, extending the computation over the ring. You were right, and the correction runs deeper than we allowed. That recomputing the ring removes the imprint we had already measured, and the paragraph above confirms it survives the rewriting. What we wrote in addition was that the same change removes most of the saving, and that we had found no intermediate that was both cheap and clean. That was a statement about our kernels, not about the scheme.

Open. NG5, at 116 000 surface nodes per GPU, is still queued; it is the point where the divergence form's lead should be largest if the ordering above is real.

All numbers are from the runs described in `danilov_mevp_report.pdf`; the output and job scripts can be shared.